STATS 195 - Final Report

AJ Fong 006513485

2025-10-16

Timeline

October 10th - October 24: Creation of Data Set for Transfer Players in the MLS From 2010-2025

```
head(transfer_data)
```

```
##   X          Name In_or_Out Age           Position                     Fee
## 1 1   A.J. Soares        In  22        Centre-Back                   draft
## 2 2 Adam Cristman        In  25     Centre-Forward                       ?
## 3 3 Adam Cristman       Out  25     Centre-Forward                       ?
## 4 4   Adam Moffat        In  24 Defensive Midfield                   draft
## 5 5   Adam Moffat       Out  24 Defensive Midfield                   draft
## 6 6  Alain Rochat        In  27        Centre-Back End of loanDec 31, 2010
##                      Team            Team_Joined     Team_Left Market_Value_num
## 1 New England Revolution New England Revolution   UC Berkeley               NA
## 2     Los Angeles Galaxy     Los Angeles Galaxy   D.C. United            1e+05
## 3           D.C. United              LA Galaxy   D.C. United            1e+05
## 4       Portland Timbers       Portland Timbers      Columbus            2e+05
## 5       Columbus Crew SC               Portland Columbus Crew            2e+05
## 6 Vancouver Whitecaps FC Vancouver Whitecaps FC     FC Zurich            5e+05
##   Fee_num Season Winter_Summer Fee_Type
## 1      NA   2010        Winter        1
## 2      NA   2010        Winter        2
## 3      NA   2010        Winter        2
## 4      NA   2010        Winter        1
## 5      NA   2010        Winter        1
## 6      NA   2010        Winter        4
```

Using the **rvest** package, I created a data set that contained information from the website, *transfermarkt.us*. Throughout this process, I came across many different challenges with merging data together. Since there are two different transfer seasons in soccer (winter/summer), I had to pull data from both seasons separately and merge them together. This was easy using the **rbind** command. The main issue was combining players transferring in/out of the league as well as intraleague transfers.

Some players would transfer from one team in the MLS to another team such as Atlanta FC to Dallas FC. This caused issues as their name would appear twice in the data set, one for each team. When it came to merging the data, this would cause repeats and mess up the **rbind** and **merge** functions. I discovered the solution to this issue would be to individually pull the *team_joined* and *team_left* data and find only the unique values of each one to remove the duplicates. This allowed for the merging of both data sets to include each team a player left and joined so that there were no *NA* values in those columns. After solving this issue, I used the **lapply** and **rep** functions to add the year and season that each transfer occurred.

October 25 - October 31st: Data Set Cleaning & Beginning of EDA

With the web scraping portion being completed, there were still issues regarding the values in the data set. A significant issue was the monetary symbol at the beginning of the *Fee* and *Market_value* columns. This made it so that these columns were being treated as characters rather than numeric values. To fix this, I used the **gsub** and **grepl** functions to removed these symbols. I then converted columns *Team, Position,* and *Age* into factors.

Another issue I came across was some teams having two names. For example, Chicago Fire FC was also named Chicago Fire. This initially caused issues in the EDA plot since the data was being improperly sorted. To address this issue, I used a **for loop** and **if-else** statement to properly change the names.

```
convert_value <- function(x) {
  x_clean <- gsub("€|\\s", "", x)
  x_clean <- gsub(",", ".", x_clean)
  num <- rep(NA_real_, length(x_clean))

  k_idx <- grepl("k$", x_clean, ignore.case = TRUE)
  num[k_idx] <- as.numeric(sub("k$", "", x_clean[k_idx], ignore.case = TRUE)) * 1000

  m_idx <- grepl("M$", x_clean, ignore.case = TRUE)
  num[m_idx] <- as.numeric(sub("M$", "", x_clean[m_idx], ignore.case = TRUE)) * 1e6

  plain_idx <- !(k_idx | m_idx | is.na(x_clean) | x_clean %in% c("-", ""))
  num[plain_idx] <- as.numeric(x_clean[plain_idx])

  return(num)
}

for (i in 1:nrow(transfer_data)) {
  team <- trimws(transfer_data$Team[i])
  if (team == "Chicago Fire") {
    transfer_data$Team[i] <- "Chicago Fire FC"
  } else if (team == "Houston Dynamo") {
    transfer_data$Team[i] <- "Houston Dynamo FC"
  } else if (team == "Columbus Crew") {
    transfer_data$Team[i] <- "Columbus Crew SC"
  }
}
```

Questions:

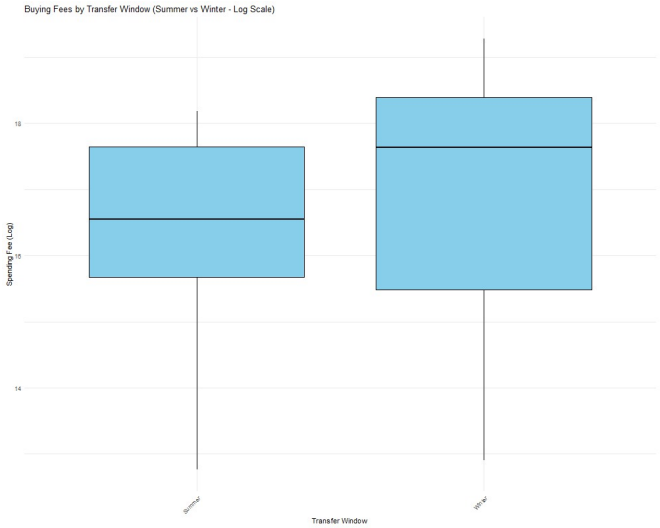
- Which teams spend/sell the most
  - Have the highest spenders/sellers changed over time?
- Is there a trend in overall spending/selling year-over-year?
  - Is there more spending in the Summer or Winter window?

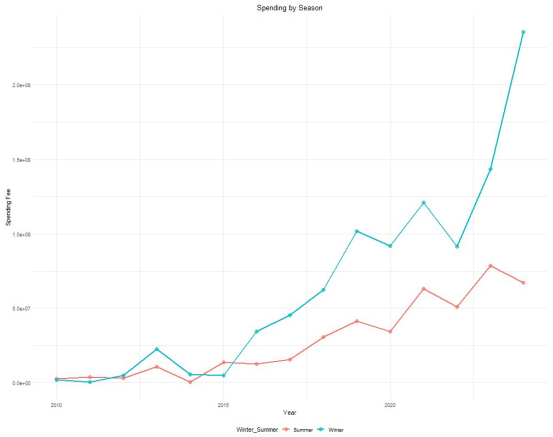
To answer these questions, I created boxplots and scatterplots to understand the relationship between spending/selling and season. I used **tidyverse** and **ggplot** to tidy, summarise, and plot the data. For the boxplots, I converted most of them to a log scale as they made it easier to visualize since the normal scale plots were very squished with right tail skews. It made the plots more difficult to compare since it appeared there was minimal difference between them.

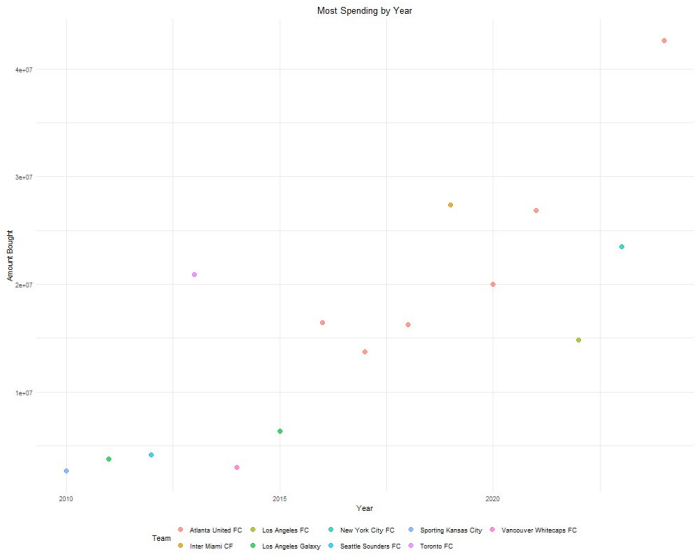
When investigating the highest sellers/spender over time, I created scatterplots to visualize the teams. The trends that I noticed was Atlanta FC being the highest spender 6 times throughout the last 9 seasons and the almost exponential growth in spending year-over-year. A point that caught my attention was the spike in 2013 spending from Toronto FC.

For season comparison, I noticed is that Winter spending is higher on average than Summer, but has more variance year-over-year. The higher variance is likely due to the sharp increase in buying during the Winter season in the last 4 years.

Below are some of the EDA plots that I created to look at these trends:





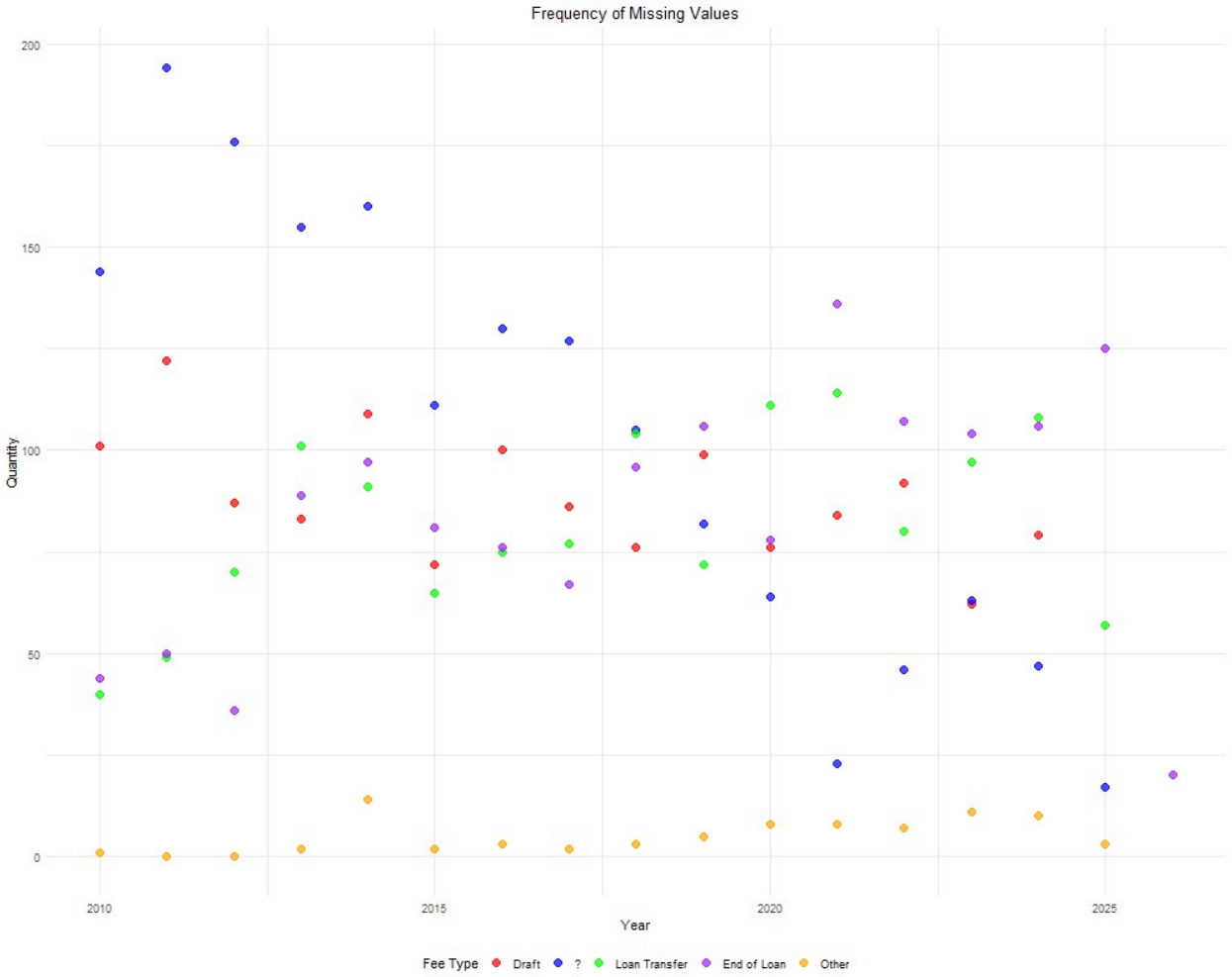


November 1st - November 14th: Full EDA & Understanding NA Values

Questions:

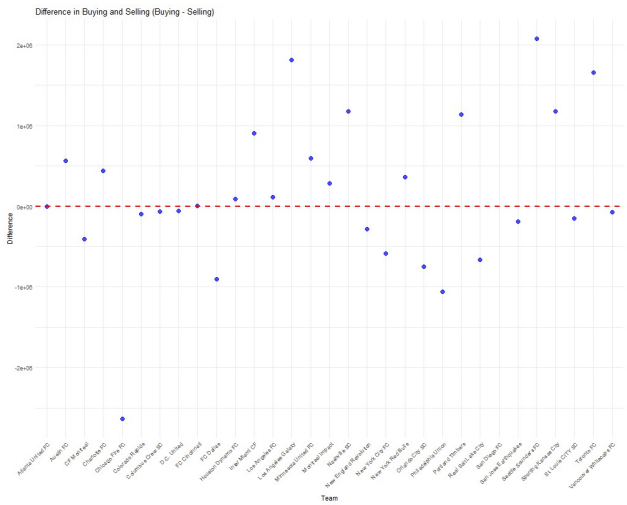
- Do teams spend more then they sell?
- Do teams spend more on younger or older players?
- Are teams spending more on attacking positions? Has there been a shift toward attacking positions vs other positions?

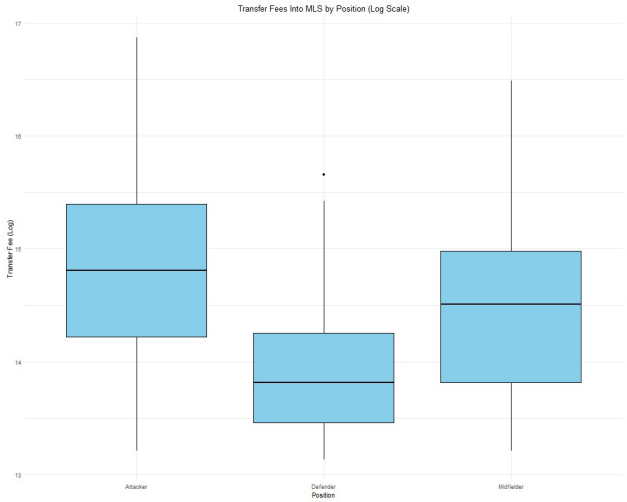
Similar to the previous section, I used **tidyverse** and **ggplot** to organize the data and create boxplots and histograms. An issue I noticed when creating the data set was the prevelence of NA values. Throughout the data, there are constant missing values in the transfer fee amounts and market value. The missing values are due to the different types of transfer transactions. For example, players who are drafted from the NCAA will not have a transfer fee since the university is not being paid. The NA values can cause issues in model and EDA optimization if there are distinct patterns being found. I created a scatterplot to investigate the issue. Ideally, there would not be a distinct pattern inside the plot.

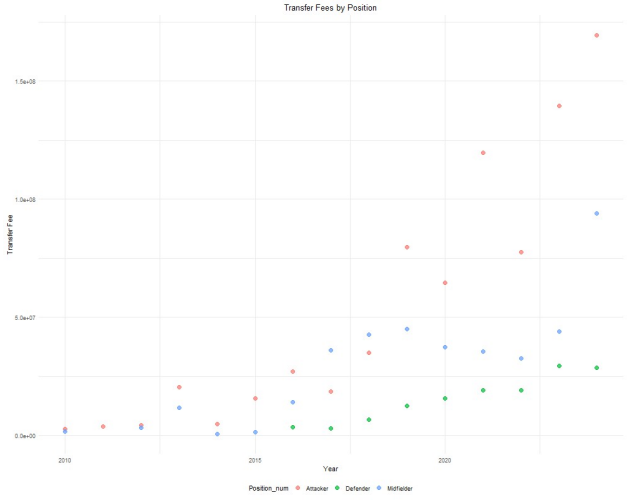


Inside the scatterplot, there appears to be a negative-linear trend in NA values from the *?* type. The other types do not matter since players being drafted directly give the reason as to why they have a NA transfer fee. To address the negative-linear trend in the *?* type, I would omit any year before 2020 when creating a model to reduce the amount of NA values there are and negate the trend.

Below are some of the EDA plots that I created to look at these trends:








```
for(i in 1:nrow(transfer_data)){
  if (transfer_data$Fee[i] == "draft"){
    transfer_data$Fee_Type[i] <- 1
  } else if (transfer_data$Fee[i] == "?"){
    transfer_data$Fee_Type[i] <- 2
```

```
  }else if (transfer_data$Fee[i] == "loan transfer"){
    transfer_data$Fee_Type[i] <- 3
  }else if (grepl("^End of loan", transfer_data$Fee[i])){
    transfer_data$Fee_Type[i] <- 4
  } else {
    transfer_data$Fee_Type[i] <- 5
  }

}
```

November 15th - December 6th: Compiling EDA Plots & Classification Model Development

Throughout these few weeks, I spent time creating a slide deck of all the EDA plots and creating a model to answer the question; When a player transfers with a fee, is there anything that drives overspending or underspending relative to the market value? Is it just age, nationality, position, or something else? For the EDA plots, I used Google Slides to create a slide deck. Each slide contained one plot along with 2-4 bullet points of observations regarding the plot. These slides culminate to show the overall trends in the data.

The model I created was a classification model that used model selection criteria methods; AIC, BIC, and LASSO Regularization, to optimize the predictors. Below is the basic logistic regression created:

```
model <- glm(overvalued ~ Age + Position + Market_Value_num + Winter_Summer, data = new_transfer_data_i
summary(model)
```

```
##
## Call:
## glm(formula = overvalued ~ Age + Position + Market_Value_num +
##     Winter_Summer, family = "binomial", data = new_transfer_data_in)
##
## Coefficients:
##                              Estimate Std. Error z value Pr(>|z|)
## (Intercept)                 6.329e+00  7.429e-01   8.520   <2e-16 ***
## Age                        -2.686e-01  2.792e-02  -9.619   <2e-16 ***
## PositionCentral Midfield   -4.967e-02  3.865e-01  -0.129   0.8977
## PositionCentre-Back        -2.914e-01  3.675e-01  -0.793   0.4278
## PositionCentre-Forward      7.238e-02  3.445e-01   0.210   0.8336
## PositionDefensive Midfield  6.507e-02  3.894e-01   0.167   0.8673
## PositionGoalkeeper         -9.513e-01  5.610e-01  -1.696   0.0899 .
## PositionLeft-Back          -7.644e-01  4.615e-01  -1.656   0.0977 .
## PositionLeft Midfield      -1.873e+00  1.271e+00  -1.473   0.1407
## PositionLeft Winger        -2.307e-01  3.835e-01  -0.602   0.5474
## PositionRight-Back         -8.175e-01  4.820e-01  -1.696   0.0899 .
## PositionRight Midfield     -1.231e+00  1.211e+00  -1.016   0.3096
## PositionRight Winger        2.962e-01  3.747e-01   0.790   0.4293
## PositionSecond Striker      2.466e-01  9.755e-01   0.253   0.8004
## Market_Value_num           -1.062e-07  4.174e-08  -2.543   0.0110 *
## Winter_SummerWinter        -2.947e-01  1.806e-01  -1.632   0.1026
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1025.78  on 813  degrees of freedom
```

```
## Residual deviance:  872.37  on 798  degrees of freedom
## AIC: 904.37
##
## Number of Fisher Scoring iterations: 4
```

From here, I implemented AIC, BIC, and LASSO with the **MASS** package to determine the predictors that would be optimal for a cross validation classification test.

```
selected_vars
```

```
##  [1] "(Intercept)"                "Age"
##  [3] "PositionCentre-Back"        "PositionCentre-Forward"
##  [5] "PositionDefensive Midfield" "PositionGoalkeeper"
##  [7] "PositionLeft-Back"          "PositionLeft Midfield"
##  [9] "PositionRight-Back"         "PositionRight Midfield"
## [11] "PositionRight Winger"       "Market_Value_num"
## [13] "Winter_SummerWinter"
```

```
summary(aic_model)
```

```
##
## Call:
## glm(formula = overvalued ~ Age + Market_Value_num + Winter_Summer,
##     family = "binomial", data = new_transfer_data_in)
##
## Coefficients:
##                        Estimate Std. Error z value Pr(>|z|)
## (Intercept)          6.105e+00  6.591e-01   9.263   <2e-16 ***
## Age                 -2.686e-01  2.678e-02 -10.030   <2e-16 ***
## Market_Value_num    -6.852e-08  3.793e-08  -1.806   0.0708 .
## Winter_SummerWinter -3.048e-01  1.765e-01  -1.727   0.0843 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1025.78  on 813  degrees of freedom
## Residual deviance:  891.13  on 810  degrees of freedom
## AIC: 899.13
##
## Number of Fisher Scoring iterations: 4
```

```
summary(bic_model)
```

```
##
## Call:
## glm(formula = overvalued ~ Age, family = "binomial", data = new_transfer_data_in)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  5.81774    0.63614   9.145   <2e-16 ***
## Age         -0.27109    0.02661 -10.186   <2e-16 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1025.78  on 813  degrees of freedom
## Residual deviance:  897.38  on 812  degrees of freedom
## AIC: 901.38
##
## Number of Fisher Scoring iterations: 4
```

These models use different information criteria to select their optimal predictors. AIC has a lighter penalty on model complexity, which makes it more flexible criteria and often leads to more predictors in pursuit of better fit. BIC has a stricter criteria on complexity which tends to lead towards a smaller model. LASSO uses a λ penalty which shrinks predictor coefficients towards 0. As λ increases, some predictors becoming zero, effectively removing them from the model. Since BIC only has one predictor, I chose to do benchmark tests on only the AIC and LASSO models.

```
### AIC ###
set.seed(123)
new_transfer_data_in_aic <- new_transfer_data_in[,c(4,10,13,15)]

tsk <- as_task_classif(new_transfer_data_in_aic, target = "overvalued")

# Learners WITH predict_type = "prob"
learners <- list(
  lrn("classif.rpart", predict_type = "prob"),
  lrn("classif.ranger", predict_type = "prob"),
  lrn("classif.log_reg", predict_type = "prob"),
  lrn("classif.kknn", predict_type = "prob")
)

# Add xgboost with one-hot encoding
learners$encode_xgb <- as_learner(
  po("encode", method = "one-hot") %>>%
    lrn("classif.xgboost", predict_type = "prob")
)

# 10-fold CV
rsmp_cv5 <- rsmp("cv", folds = 10)

# Benchmark design
bmr_design <- benchmark_grid(
  tasks = tsk,
  learners = learners,
  resamplings = rsmp_cv5
)

# Run benchmark
bmr <- benchmark(bmr_design)
```

```
## Warning: package 'mlr3fselect' was built under R version 4.4.3
```

```r
# Results table
results <- bmr$aggregate(list(msr("classif.mbrier"), msr("classif.acc")))[
  , .(learner_id, classif.mbrier, classif.acc)
]
results
```

```
##                learner_id classif.mbrier classif.acc
##                    <char>          <num>       <num>
## 1:          classif.rpart      0.3979394   0.7174496
## 2:         classif.ranger      0.3681879   0.7235170
## 3:        classif.log_reg      0.3688712   0.7283951
## 4:           classif.kknn      0.4140811   0.6891900
## 5: encode.classif.xgboost      0.5185364   0.6609304
```

```r
### LASSO ###
set.seed(123)
new_transfer_data_in_lasso <- new_transfer_data_in[,c(4,5,10,13,15)]

tsk <- as_task_classif(new_transfer_data_in_lasso, target = "overvalued")

# Learners WITH predict_type = "prob"
learners <- list(
  lrn("classif.rpart", predict_type = "prob"),
  lrn("classif.ranger", predict_type = "prob"),
  lrn("classif.log_reg", predict_type = "prob"),
  lrn("classif.kknn", predict_type = "prob")
)

# Add xgboost with one-hot encoding
learners$encode_xgb <- as_learner(
  po("encode", method = "one-hot") %>>%
    lrn("classif.xgboost", predict_type = "prob")
)

# 10-fold CV
rsmp_cv5 <- rsmp("cv", folds = 10)

# Benchmark design
bmr_design <- benchmark_grid(
  tasks = tsk,
  learners = learners,
  resamplings = rsmp_cv5
)

# Run benchmark
bmr <- benchmark(bmr_design)
```

```r
# Results table
results <- bmr$aggregate(list(msr("classif.mbrier"), msr("classif.acc")))[
  , .(learner_id, classif.mbrier, classif.acc)
]
results
```

```
##                learner_id classif.mbrier classif.acc
##                    <char>          <num>       <num>
## 1:          classif.rpart      0.3938674   0.7198735
## 2:         classif.ranger      0.3620484   0.7334237
## 3:        classif.log_reg      0.3760088   0.7333785
## 4:           classif.kknn      0.4087150   0.7174797
## 5: encode.classif.xgboost      0.5095690   0.6965673
```

With the **mlr3verse** package, I used 10-fold cross-validation and benchmarked several classification models such as regression trees, random forests, logistic regression, K-Nearest Neighbors, and XGBoost. I used these to identify the best predictor of whether a player is overvalued. The LASSO-regularize random forest had the strongest performance with an accuracy of 73.3%. This indicates that the regression tree provides a reasonably strong level of predictive power.